Task 1

1. Introduction

This report provides a comprehensive analysis of a C++ program designed to simulate a Unix-like shell environment. The program provides a command-line interface that allows users to execute built-in commands, manage environment variables, handle file I/O redirection, and execute external system programs using process management system calls.

2. Detailed Function Analysis

2.1 Save IO:

```
//to create a copy of the current input/output mode
void saveIO(int &saved_in, int &saved_out) {
    saved_in = dup( fd: STDIN_FILENO);
    saved_out = dup( fd: STDOUT_FILENO);
}
```

Purpose: This function creates a backup of the current standard input and standard output file descriptors.

Logic:

- It uses the dup() system call, which creates a copy of a file descriptor.
- STDIN_FILENO (0) and STDOUT_FILENO (1) are duplicated and stored in the reference parameters.

Why it's needed: When the shell redirects output to a file, it overrides the terminal's output. This function ensures that the shell can reverse back to normal mode by taking the input from the user and print it normally not in a file.

2.2 Restore IO:

```
//switch back to default input mode
void restoreIO(int saved_in, int saved_out) {
    if (saved_in != -1) {
        dup2( fd: saved_in,  fd2: STDIN_FILENO);
        close(saved_in);
    }
    if (saved_out != -1) {
        dup2( fd: saved_out,  fd2: STDOUT_FILENO);
        close(saved_out);
    }
}
```

Purpose: This function restores the shell's input and output back to the original state (the terminal).

Logic:

- It uses dup2(), which forces a specific file descriptor to point to the same location as another.
- It redirects saved_in back to STDIN_FILENO and saved_out back to STDOUT_FILENO.
- Finally, it closes the backup descriptors to prevent memory leaks.

2.3 Apply Redirection

```
bool applyRedirection(const string &inputFile, const string &outputFile, bool appendMode) {
    if (!inputFile.empty()) {
        int fd = open( file: inputFile.c_str(), oflag: O_RDONLY);
        if (fd < 0) {
            perror( s: "Input file error");
            return false;
        }
        dup2(fd, fd2: STDIN_FILENO);
        close(fd);
    }

    if (!outputFile.empty()) {
        int flags = O_WRONLY | O_CREAT;
        flags |= (appendMode ? O_APPEND : O_TRUNC);

        // 0644 gives read/write to owner, read to others
        int fd = open( file: outputFile.c_str(), flags, 0644);
        if (fd < 0) {
            perror( s: "Output file error");
            return false;
        }
        dup2(fd, fd2: STDOUT_FILENO);
        close(fd);
    }
    return true;
}
```

Purpose: This function handles the logic for <, >, and >> operators.

Logic:

- **Input Redirection (<):** Opens the specified file in read-only mode (O_RDONLY). If successful, it uses dup2 to replace STDIN with the file's descriptor.
- **Output Redirection (> and >>):** If append Mode is false, it uses O_TRUNC to overwrite the file.
    - If append Mode is true, it uses O_APPEND to add to the end of the file.
    - It uses O_CREAT with permissions 0644 (owner can read/write, others can read) to create the file if it doesn't exist.
- **Return:** Returns true if redirection was successful, or false (with an error message) if a file could not be opened.

3. The Main Loop and Command Processing

The main function serves as the heart of the shell, operating in a continuous while loop.

3.1 Shell Initialization and Batch Mode

```
int main(int argc, char *argv[]) {
    string command;
    //the default input mode is cin
    istream *input_source = &cin;
    ifstream file_stream;
    int saved_stdin = -1;
    int saved_stdout = -1;
    //if there is a batch file provided switch to file
    if (argc > 1) {
        file_stream.open(argv[1]);
        if (!file_stream.is_open()) {
            cout << "Error: Could not open file " << argv[1] << endl;
            return 1;
        }
        input_source = &file_stream;
    }
```

- The shell checks argc. If an argument is provided, it tries to open that file as an ifstream and redirects the input source from cin to the file.
- getcwd(cwd, sizeof(cwd)) is called in every iteration to retrieve the current working directory, which is displayed in the prompt.

3.2 Command Tokenization (Parsing)

```
stringstream ss(command);
string word;
vector<string> words;
bool backgroundRunning = false;

// loop to split the command by spaces
while (ss >> word) {
    // skipping the spaces
    words.push_back(word);
}
//check if there is & at the end
if (words.back() == "&") {
    backgroundRunning = true;
    words.pop_back();
}
```

- The shell reads a line using getline().
- It uses stringstream to split the string into a vector of strings named words. This separates the command from its arguments by whitespace.
- It checks for the ampersand symbol at the end of the command. If found, backgroundRunning is set to true, and the ampersand is removed from the arguments.

3.3 Redirection Parsing

```
for (int i = 0; i < words.size(); i++) {
    if (words[i] == "<") {
        if (i + 1 < words.size()) {
            inputFile = words[i + 1];
            // Remove "<" and "file"
            words.erase( first: words.begin() + i,  last: words.begin() + i + 2);
            //update the index since elements are removed
            i--;
        } else {
            cout << "Error: No input file specified.\n";
        }
```

The shell iterates through the tokens to find redirection symbols:

- <: Identifies the next token as the input source.

```
} else if (words[i] == ">") {
    if (i + 1 < words.size()) {
        outputFile = words[i + 1];
        appendMode = false;
        words.erase( first: words.begin() + i,  last: words.begin() + i + 2);
        i--;
    } else {
        cout << "Error: No output file specified.\n";
    }
} else if (words[i] == ">>") {
    if (i + 1 < words.size()) {
        outputFile = words[i + 1];
        appendMode = true;
        words.erase( first: words.begin() + i,  last: words.begin() + i + 2);
        i--;
    } else {
        cerr << "Error: No output file specified.\n";
    }
}
```

- >: Identifies the next token as the output destination (overwrite).
- >>: Identifies the next token as the output destination (append).
- Once identified, the symbols and filenames are erased from the words vector, so they aren't passed as arguments to the actual command.

3.4 Built-in Command Implementation

The shell manually handles several commands:

1. cd:

```
if (words[0] == "cd") {
    if (words.size() > 1) {
        char *path = words[1].data(); //to make it C style
        if (chdir(path) != 0) {
            cout << "Error the directory does not exist\n";
        } else {
            getcwd(cwd, size: sizeof(cwd));
            setenv( name: "PWD", value: cwd, replace: 1);
        }
    } else
        cout << cwd << "\n";
```

- Accpets a directory name from the user.
- Print an error if the directory does not exist.
- Changes the PWD directory in the using the setenv().

2. dir:

```
} else if (words[0] == "dir") {
    int saved_in = -1, saved_out = -1;
    saveIO([&]saved_in, [&]saved_out); //save the current mode
    //check if there will be mood change
    if (applyRedirection(inputFile, outputFile, appendMode)) {
        //check if there is a directory name entered
        const char *path = (words.size() > 1) ? words[1].c_str() : ".";
        DIR *dir = opendir(path);
        if (!dir) {
            //no directory with the entered name
            perror(s:"dir");
        } else {
            //a pointer to the data inside the current directory
            dirent *entry;
            while ((entry = readdir(dir)) != nullptr) {
                cout << entry -> d_name << " ";
            }
            cout << endl;
            closedir(dir);
        }
    }
    //switch the mode back to normal
    restoreIO(saved_in, saved_out);
```

- Performs a check if the user enters a directory name.
    - If no directory, then the shell opens the current directory.
    - If there is a directory, then the shell tries to open that directory.
- After the directory is opened, a pointer to the directory content is initialized (dirent).
- A loop starts to print the directory content.
- The directory is then closed.

3. environ:

```
} else if (words[0] == "environ") {
    int saved_in = -1, saved_out = -1;
    saveIO( [&] saved_in, [&] saved_out);

    if (applyRedirection(inputFile, outputFile, appendMode)) {
        //printing all the environment variables
        for (char **env = environ; *env != 0; env++) {
            cout << *env << endl;
        }
    }
    restoreIO(saved_in, saved_out);
```

- A loop is made on all the environment variables from the global pointer.

4. set:

```
} else if (words[0] == "set") {
    if (words.size() < 3) {
        cerr << "Error: usage is 'set VARIABLE VALUE'\n";
    } else {
        setenv( name: words[1].c_str(), value: words[2].c_str(), replace: 1);
    }
```

- using the setenv(), it accepts a variable name and value.
- The variable data is changed to the value.
- An error message is shown if there is missing values.

5. echo:

```
} else if (words[0] == "echo") {
    int saved_in = -1, saved_out = -1;
    saveIO([&]saved_in, [&]saved_out);

    if (applyRedirection(inputFile, outputFile, appendMode)) {
        for (size_t i = 1; i < words.size(); i++) {
            cout << words[i] << (i == words.size() - 1 ? "" : " ");
        }
        cout << "\n";
    }
    restoreIO(saved_in, saved_out);
```

- Print the entered string in the terminal

6. help:

```
else if (words[0] == "help") {
  int saved_in = -1, saved_out = -1;
  saveIO([&]saved_in, [&]saved_out);

  if (applyRedirection(inputFile, outputFile, appendMode)) {
      if (words.size() == 1) {
          cout << "This is a terminal simulation system that has support to few commands including:\n"
                  << "cd\n"
                  << "dir\n"
                  << "environ\n"
                  << "set\n"
                  << "echo\n"
                  << "help\n"
                  << "pause\n"
                  << "to know more about a command enter help and the command \n";
      } else if (words[1] == "cd")
          cout << "The cd functions changes the current directory.\n"
                  << "to use it, enter cd followed by a space and the directory name you need to move to\n";
```

- Displays a main message explaining the commands handled by the terminal
- Can accept a string containing the command and the shell will print how to use the command

7. pause:

```
} else if (words[0] == "pause") {
    cout << "Shell is paused press enter to resume";
    string buffer;
    getline( [&] cin, [&] buffer);
```

- Uses getline() to freeze the shell until the user presses enter.

8. quit:

- Breaks the loop to terminate the program.

3.5 External Command Execution (Process Management)

If the command is not a built-in, the shell attempts to run it as an external program:

1. **fork ():** Creates a child process.

- **Child Process (PID == 0)**:

```
else {
  pid_t pid = fork();
  if (pid < 0)
      cout << "Fork Failed\n";
  else if (pid == 0) {
      vector<char *> args;
      for (auto &s :string & : words) args.push_back( x: s.data());
      args.push_back( x: nullptr);
      //check if there is an input file
      if (!applyRedirection(inputFile, outputFile, appendMode)) {
          exit( status: 1);
      }
      execvp(args[0], argv: args.data());
      perror( s: "Error in executing command\n");
      exit( status: 1);
```

- Converts the vector of strings into a C-style array of char*.
- Applies any requested I/O redirection.
- Calls execvp(), which searches the system's PATH for the executable and replaces the child process with it.

- **Parent Process (PID > 0)**:

```
else {
  if (backgroundRunning) {
      cout << "The process is running in the background, pid: " << pid << "\n";
  } else {
      int status;
      //the waiting process
      //the status to know how the process got terminated
      //and can be null if we dont care about the process
      //options to show how to wait either freeze or not
      waitpid(pid, &status, options: 0);
  }
```

- If backgroundRunning is false, it calls waitpid() to wait for the child to finish.
- If backgroundRunning is true, it prints the child's PID and immediately returns to the prompt, letting the child run asynchronously.

5. Github:

Repo link: https://github.com/youssef1316/OS_coursework1

Commit sha: d37090d